Cyber Security Internship | Task 01

Vulnerability Assessment Report

Web Application Security Audit | Passive Read-Only Scope

Target	demo.testfire.net	**Assessment Type**	Passive / Read-Only	**Total Findings**	7 Vulnerabilities
Prepared By	Valery Vezenegho	**CIN ID**	FIT/MAR26/CS7097	**IP Address**	65.61.137.117

Executive Summary

A passive security assessment was conducted on demo.testfire.net, an intentionally vulnerable demo banking application, on 21 April 2026. Using industry-standard tools the assessment identified 7 security vulnerabilities across multiple categories.

The most critical findings are the absence of essential HTTP security headers and the lack of HTTPS enforcement. Both can be resolved within a single working day through server configuration changes at no cost. No exploitation or active attacks were performed — all testing was strictly read-only and passive.

2	4	1
HIGH RISK	MEDIUM RISK	LOW RISK

#	FINDING	RISK LEVEL	TOOL USED
1	Missing Security Headers	HIGH	SecurityHeaders.com / DevTools
2	HTTP Without HTTPS Redirect	HIGH	SecurityHeaders.com / Nmap
3	Server Version Publicly Disclosed	MEDIUM	Nmap
4	Port 8080 Unnecessarily Exposed	MEDIUM	Nmap
5	Weak SSL/TLS Configuration	MEDIUM	SSL Labs
6	Absence of Anti-CSRF Tokens	MEDIUM	OWASP ZAP
7	Developer Comments in Page Source	LOW	OWASP ZAP

Scope & Tools

FIELD	DETAILS
Target Website	demo.testfire.net
IP Address	65.61.137.117
Application	Altoro Mutual — IBM Demo Banking Application
Assessment Type	Passive — Read-Only
Date	21 April 2026
Assessor	Valery Vezenegho

Tools Used

TOOL	VERSION	PURPOSE
Nmap	7.95	Port scanning and HTTP header analysis
OWASP ZAP	2.17.0	Passive vulnerability scanning via proxy
Firefox DevTools	—	Response header inspection
SecurityHeaders.com	—	Automated security header analysis
SSL Labs (Qualys)	—	SSL/TLS configuration analysis

Scope Boundaries

✓ PERMITTED ACTIVITIES	✕ PROHIBITED ACTIVITIES
✓ Public-facing pages only ✓ Passive scanning ✓ HTTP header checks ✓ Configuration analysis ✓ SSL/TLS analysis	✕ Login bypass or credential attacks ✕ Exploitation of vulnerabilities ✕ Brute force attacks ✕ Denial of service ✕ Any activity that harms the site

Identified Vulnerabilities

FINDING 01

Missing Security Headers

▲ HIGH RISK

What is the issue

The website is missing critical HTTP security headers — configuration instructions that tell browsers how to handle website content. Without them, several attack categories become possible including script injection, clickjacking, and session hijacking.

Missing Headers Identified

MISSING HEADER	RISK	WHAT ATTACKERS CAN DO WITHOUT IT
Content-Security-Policy	HIGH	Inject malicious scripts into pages (XSS)
Strict-Transport-Security	HIGH	Force users onto unencrypted HTTP connections
X-Frame-Options	MEDIUM	Embed the site in fake pages to trick users (Clickjacking)
X-Content-Type-Options	LOW	Cause browser to misinterpret file types
Referrer-Policy	LOW	Expose user navigation paths to third parties
Permissions-Policy	LOW	Access browser features without restriction

What This Means For Your Business

Without these headers, every visitor to the website is exposed to attacks that can silently steal login credentials and personal data.

How to Fix It

Add all missing headers to the web server configuration. This is a 30-minute configuration change — no code changes required.

FINDING 02

HTTP Without HTTPS Redirect

▲ HIGH RISK

What is the issue

The website serves content over HTTP which is completely unencrypted. It does not automatically redirect users to the secure HTTPS version. All data transmitted between users and the server — passwords, session tokens, personal information — travels as plain readable text.

What This Means For Your Business

Any user on a public WiFi network — a coffee shop, airport, or hotel — could have their login credentials stolen in real time by anyone on the same network. This requires no hacking skill, only freely available software.

How to Fix It

Configure the server to redirect all HTTP traffic to HTTPS automatically. Add an HSTS header to lock this in permanently. Implementation takes less than one hour.

FINDING 03

Server Software Version Publicly Disclosed

● MEDIUM RISK

What is the issue

The server includes its exact software name and version — Apache-Coyote/1.1 — in every HTTP response it sends. Apache-Coyote is an outdated component of Apache Tomcat. Attackers can use this version information to search for and target known vulnerabilities specific to that version.

What This Means For Your Business

Disclosing server software versions is like labelling your security system on your front door. It removes a layer of obscurity and gives attackers a roadmap before they have even started probing.

How to Fix It

Configure the server to suppress or replace the Server header. Approximately 30 minutes to implement.

FINDING 04

Port 8080 Unnecessarily Exposed

● MEDIUM RISK

What is the issue

Port 8080 is open and running the same web application as the main site on port 80. Critically, the session cookie on port 8080 is missing the Secure flag — meaning session data can be transmitted over an unencrypted connection. Every unnecessary open port increases the attack surface.

How to Fix It

Close port 8080 if not required. If it must remain open, apply the same security configuration including Secure and SameSite cookie flags.

FINDING 05

Weak SSL/TLS Configuration

● MEDIUM RISK

What is the issue

SSL Labs rated the overall SSL configuration as Grade B. The server supports outdated protocols TLS 1.0 and TLS 1.1 which have known vulnerabilities and have been retired by all major standards bodies since 2021. The most secure current version, TLS 1.3, is not supported at all. Multiple cipher suites are also marked as weak.

SSL/TLS ISSUE	RISK
TLS 1.0 supported — deprecated and vulnerable	HIGH
TLS 1.1 supported — deprecated and vulnerable	HIGH
TLS 1.3 not supported — latest security standard missing	MEDIUM
Weak Diffie-Hellman key exchange parameters	MEDIUM
Multiple weak cipher suites active	MEDIUM
SSL certificate expires June 2026 — renewal needed soon	LOW

How to Fix It

Disable TLS 1.0 and TLS 1.1. Enable TLS 1.3. Remove weak cipher suites. Renew the SSL certificate before June 2026.

FINDING 06

Absence of Anti-CSRF Tokens

MEDIUM RISK

What is the issue

OWASP ZAP detected that web forms on the application do not use Anti-CSRF (Cross-Site Request Forgery) tokens. Without these tokens, an attacker can trick a logged-in user into unknowingly submitting forms — such as transferring funds or changing account settings — simply by getting them to visit a malicious page in another browser tab.

What This Means For Your Business

An authenticated banking customer could have transactions executed on their account without their knowledge or consent just by visiting a malicious external link.

How to Fix It

Implement CSRF tokens on all forms that perform actions. Most web frameworks include built-in CSRF protection that can be enabled through configuration settings.

FINDING 07

Developer Comments Visible in Page Source

LOW RISK

What is the issue

OWASP ZAP identified suspicious developer comments left in the publicly visible page source code. These comments can reveal information about the application's internal structure, logic, and configuration that was intended to remain private. Attackers routinely read page source before launching targeted attacks.

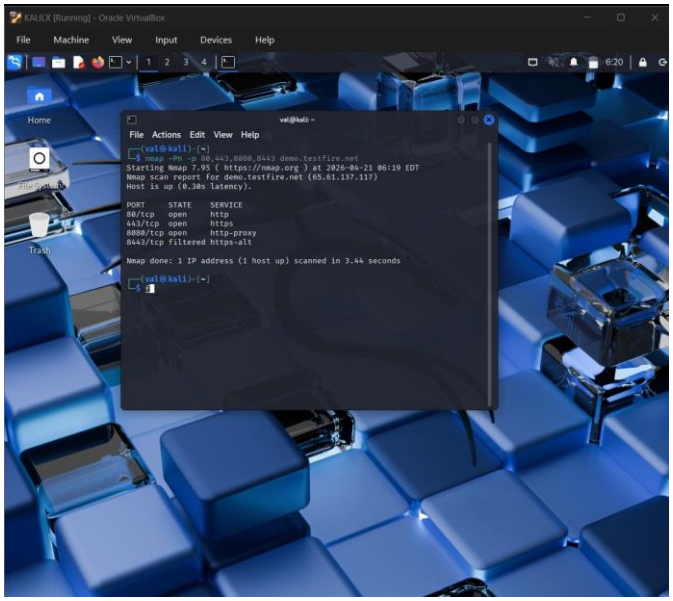
How to Fix It

Remove all developer comments from production code before deployment. Use an automated build process that strips comments automatically.

Tool Output Screenshots

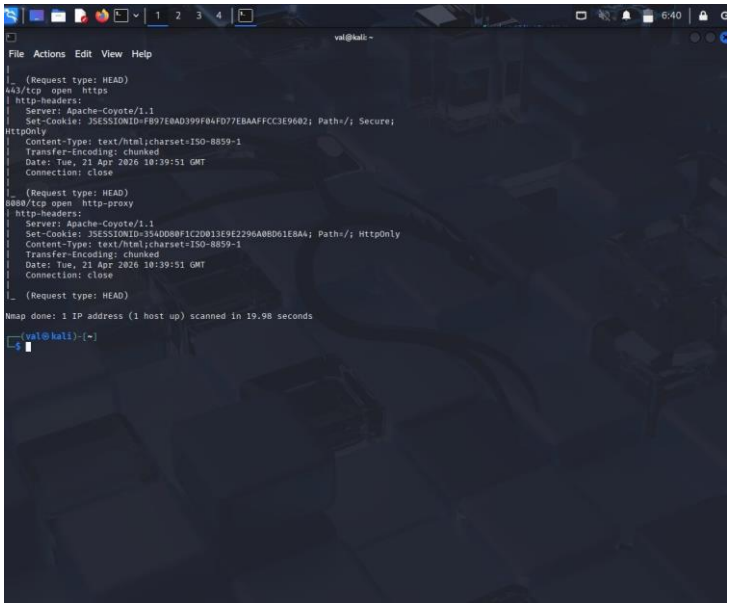
The following screenshots were taken during the assessment in the order they were captured. Each shows the raw output from the security testing tools used.

Screenshot 1 | Nmap Port Scan



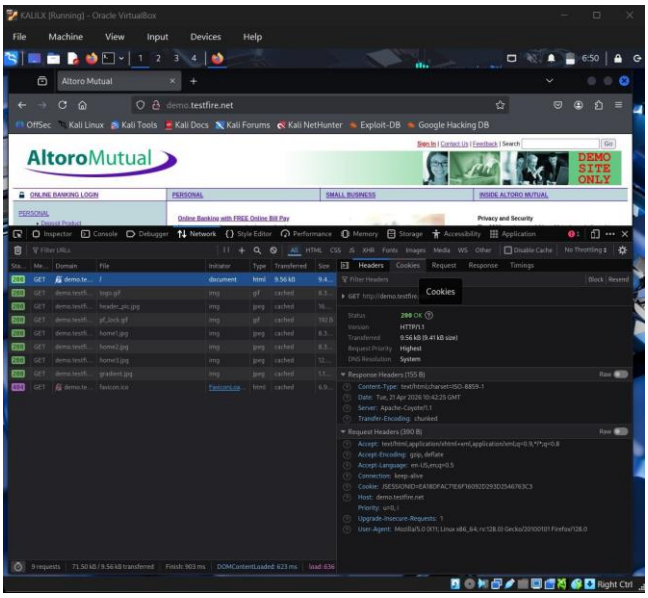
What this proves: Confirms that ports 80, 443, and 8080 are all open and active. Port 8080 being open is Finding 4 — an additional HTTP entry point that should be closed if not needed. The host resolves to IP 65.61.137.117.

Screenshot 2 | Nmap http-headers Script



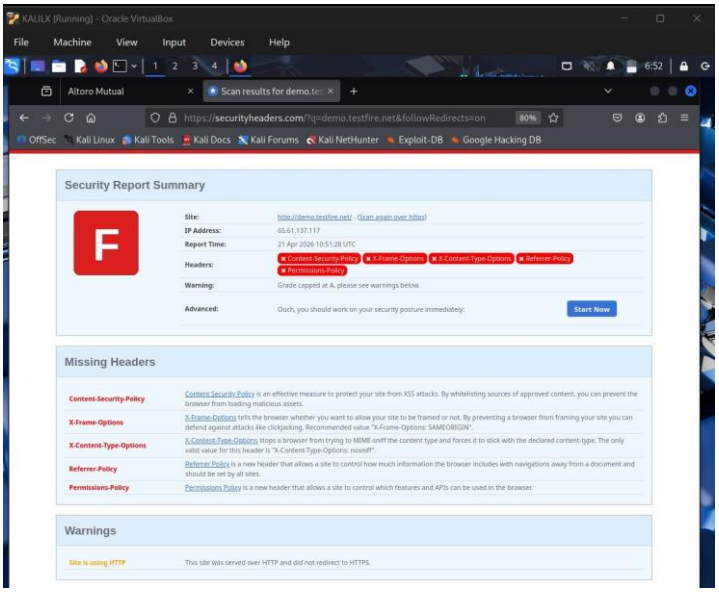
What this proves: Directly confirms Finding 3 — Apache-Coyote/1.1 is exposed on both ports 443 and 8080. Also confirms Finding 4 — port 8080 session cookie is missing the Secure flag, meaning session tokens can be transmitted over unencrypted connections.

Screenshot 3 | Firefox DevTools Response Headers



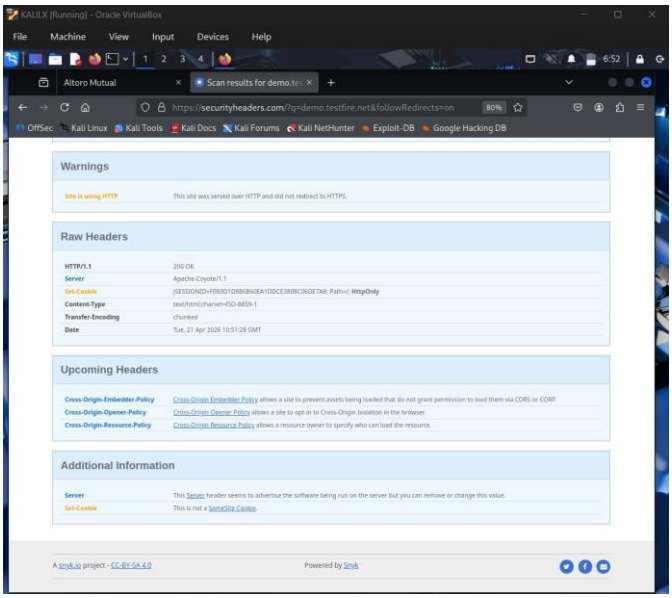
What this proves: Confirms the site loads over unencrypted HTTP and returns only 4 response headers. All critical security headers — Content-Security-Policy, X-Frame-Options, Strict-Transport-Security, X-Content-Type-Options, Referrer-Policy, and Permissions-Policy — are completely absent. This directly evidences both Finding 1 and Finding 2.

Screenshot 4 | SecurityHeaders.com Scan Result



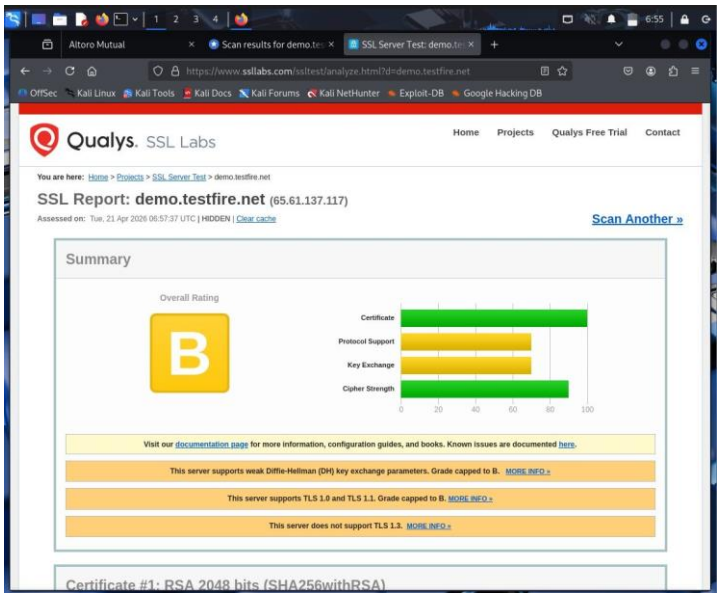
What this proves: The F grade directly confirms Finding 1 and Finding 2. Five security headers are completely absent and the site has no HTTPS enforcement — the two most critical configuration failures identified in this assessment.

Screenshot 5 | SecurityHeaders.com Raw Headers



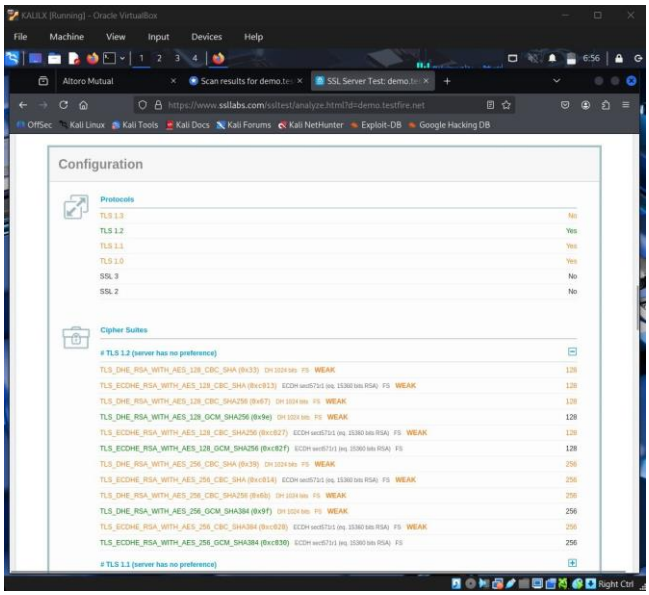
What this proves: Confirms that only 5 basic headers are present out of the 11 that should be configured. The Apache-Coyote/1.1 server disclosure is identified as Finding 3, and the missing SameSite cookie attribute is an additional session security risk.

Screenshot 6 | SSL Labs Overall Grade



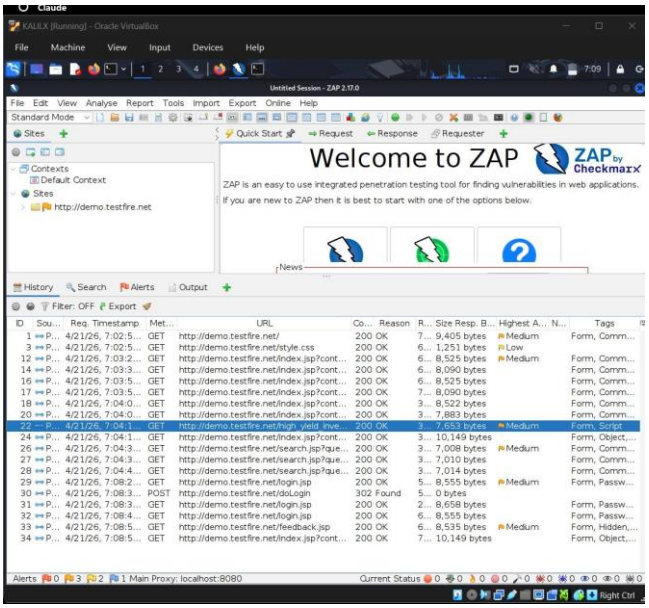
What this proves: Confirms Finding 5 — the SSL configuration has significant weaknesses. The grade is capped at B specifically because of weak DH parameters and legacy TLS protocol support. The absence of TLS 1.3 means the server cannot use the most secure available encryption protocol.

Screenshot 7 | SSL Labs Protocol & Cipher Configuration



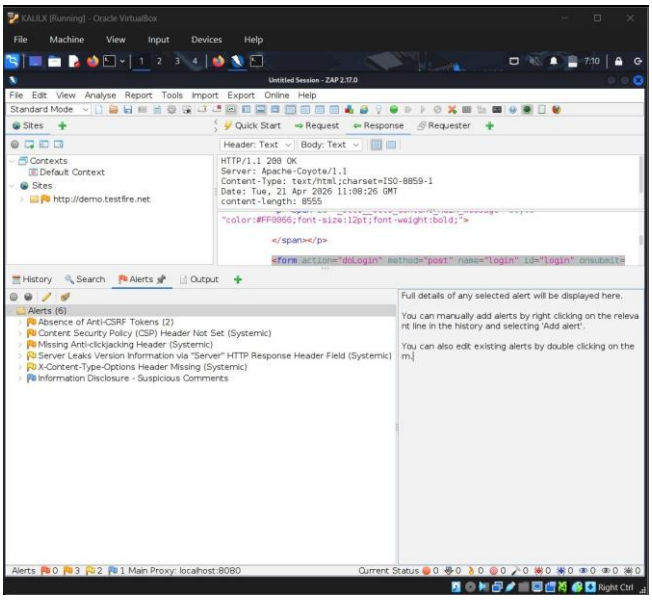
What this proves: Directly confirms that TLS 1.0 and TLS 1.1 are both enabled despite being deprecated. TLS 1.3 is completely absent. The cipher suite list shows that most available encryption options are rated WEAK — meaning encrypted connections may be vulnerable to downgrade attacks.

Screenshot 8 | OWASP ZAP Sites Panel



What this proves: Confirms that OWASP ZAP successfully captured traffic from all major areas of the application during passive browsing — including the login form, search functionality, and financial pages. The medium risk flags visible in the request list confirm vulnerabilities were detected across multiple pages, not just the homepage.

Screenshot 9 | OWASP ZAP Alerts



What this proves: Confirms Findings 6 and 7 directly from ZAP passive scanning. The Anti-CSRF finding appears twice because it was detected on multiple forms. The server version leak and missing headers confirm Findings 1 and 3, and the Anti-Clickjacking alert matches the X-Frame-Options finding identified in SecurityHeaders.com.

Recommendations

The following actions are recommended in order of priority. All Immediate items can be completed within a single working day at no additional cost.

PRIORITY	ACTION	EFFORT
IMMEDIATE	Enable HTTPS redirect and add HSTS header	~1 hour
IMMEDIATE	Add all missing security headers to server configuration	~1 hour
THIS WEEK	Disable TLS 1.0 and TLS 1.1, enable TLS 1.3	~Half a day
THIS WEEK	Remove weak cipher suites from SSL configuration	~Half a day
THIS WEEK	Suppress server version information from HTTP headers	~30 minutes
THIS MONTH	Implement CSRF tokens on all state-changing forms	1–2 days
THIS MONTH	Remove developer comments from production source code	~2 hours
BEFORE JUNE	Renew SSL certificate before expiration in June 2026	~1 hour

Conclusion

The passive assessment of demo.testfire.net identified 7 security vulnerabilities — 2 high risk, 4 medium risk, and 1 low risk — covering missing security headers, unencrypted connections, an outdated server component, weak SSL configuration, exposed ports, absent CSRF protection, and information disclosure through developer comments.

None of these findings require sophisticated techniques to exploit. The two highest-risk issues — missing security headers and lack of HTTPS enforcement — can be resolved with a single working day through server configuration changes at no cost. With all recommended remediations applied, the security posture of this application would improve significantly.

Regular passive security assessments are recommended on a quarterly basis to maintain a strong security baseline as the application and its underlying infrastructure continue to evolve.